The six measurements of slide 24

Slide 24 lists six things to measure on the ZedBoard to prove the claims of v2. For each one this guide says how to measure it, what the simulation already shows, and where to put the board result. Fill in the "board" column when you have run it; keep the simulation column as the reference.

#	what	simulation result	board result
1	flexibility: one bitstream runs UART, SPI, I2C and one new protocol	all four protocols pass as programs (`tb_se_programs`, 69 checks)	
2	speed: 115200 baud, I2C 100 / 400 kHz, top SPI clock	8680.0 ns per UART bit (115200.0 baud), SCL high/low 5 us (100 kHz)	
3	switch time, request to first new edge	2 clocks (20 ns) from an idle engine; a busy engine is waited for (39 325 clocks for the rest of an SPI frame)	
4	CPU load per reading	v1: 10 accesses + 2 IRQs; v2: 0 accesses + 1/32 IRQ (`tb_ssc2_system`)	
5	area per block	yosys estimate below	
6	NoC scalability: bus vs crossbar vs NoC at 4, 8, 16 nodes	`noc_study/results.md`	

---

1. Flexibility

Claim: a new protocol is a program, not a new bitstream.

How: with the Way 2 bitstream and app, run these console commands one after the other, without reprogramming the FPGA:

```
se uart     -> HELLO FROM THE SERIAL ENGINE, 0 errors, PASS
se spi      -> READ ID -> 20 ba 19
se i2c      -> TMP2 -> 0x0c 0x80 = +25.0000 C
se man      -> 32 bytes, 0 errors, CRC equal, PASS       (the new protocol)
```

Report: the four results, and that the bitstream was the same for all of them. Also report the program sizes (`programs/*.lst`): uart_tx 7 words, uart_rx 11, spi_master 14, i2c_master 32, manchester_tx 13, manchester_rx 12, oled_spi 6, and the time to load one (32 register writes, a few microseconds over AXI).

2. Speed

Use a logic analyser (8 channels, at least 24 MHz sample rate; PulseView / sigrok with a cheap FX2 analyser is enough) on JD.

what	set-up	what to read off
UART 115200 (fixed engine)	probe JD7 (copy of the UART engine's TXD) while `flowa` + `pc_frame.py COMx 01 9F 03` returns the flash ID	bit time 8.68 us; decodes as UART 115200 8N1: 20 BA 19
UART 115200 (program)	`switch` or `se uart`, probe JD2	bit time 8.68 us (simulation: 8680.0 ns)
I2C 100 kHz	`temp`, probe JD9 (SCL) and JD10 (SDA)	SCL period 10 us; decode I2C: 96 00 / 97 0C 80
I2C 400 kHz	in `sw/main.c` change `ssc_i2c_init(100000u)` to `400000u` (prescale 62), run `temp`	SCL period about 2.5 us (v1 measured 383 kHz with the board pull-ups)
top SPI clock, fixed engine	`ssc_spi_init()` with 12.5 MHz (CLKDIV 3), `id`	the highest rate at which the ID still reads 20 BA 19
top SPI clock, program	change `.clkdiv 25 0` in `programs/spi_master.se` to 12, 6, 3 ... rebuild the programs and the app, run `se spi`	SCK = 100 MHz / (4 x INT); the highest INT that still reads 20 BA 19

The programmable engine needs 4 ticks per SPI bit and its inputs pass a 2-clock synchroniser, so expect it to stop before the fixed engine (slide 22: "the programmable engine may not reach the fixed SPI engine's top clock"). Report both numbers.

3. Switch time

How: console command `switch`. It routes JD to the SPI engine, starts a 16-byte SPI frame, asks the crossbar to give JD to SM0 *during* the frame, and prints:

```
JD while the SPI frame runs: still SPI, switching=1
JD now: SM0.  SW_CYCLES = ~30000 clocks (~300 us, it waited for the SPI frame)
SM0 sent "HELLO FROM SM0 ON JD" on JD2 (115200 8N1).  SW_EDGE = ... clocks
switch from the idle SM0 back to SPI: SW_CYCLES = 2 clocks (20 ns)
```

- `SW_CYCLES` = clocks from the request to the hand-over. From an idle engine it is 2 clocks (20 ns) in simulation; from a busy engine it is the rest of the frame (the crossbar never cuts a frame).
- `SW_EDGE` = clocks from the request to the first edge driven by the new engine. It includes the time until software gives the program its first byte.

On the logic analyser (JD1-4) you see the SPI frame finish completely, the pins float for 10 ns, then SM0's idle-high TXD and its frame on JD2.

4. CPU load per reading

How: the controller counts its own register accesses (`APB_COUNT`, 0xA10) and interrupts (`IRQ_COUNT`, 0xA14), so the CPU does not have to.

version	console	what it prints	simulation
v1 way (CPU does every step)	`temp`	`CPU load of this one reading: N APB accesses, 2 interrupts`	10 accesses + 2 IRQs (no ISR); the C driver with its ISR adds 2 accesses per interrupt
v2 way (hub + DMA)	`hub`	`CPU load for 32 readings: 2 APB accesses (2 of them in the ISR), 1 interrupt(s)`	0 accesses + 1 IRQ for 32 readings

At 10 readings per second this is 20 interrupts per second (v1) against 10/32 = 0.31 (v2): 64 times fewer, as slide 20 expects. The v2 ISR makes 2 accesses (read and clear INT2_STATUS) per batch, and consuming a batch is 1 more write (DMA_RD_IDX): about 0.1 accesses per reading.

5. Area

How: build either bitstream; Vivado writes `build/ps_system/utilization_hier.rpt` (or `build/standalone/...`), with LUTs, FFs and BRAM per block. Copy the rows for `u_noc` (6 routers), `u_se`, `u_xbar`, `u_hub`, `u_dma`, `u_ni_*`, `u_uart`, `u_spi`, `u_i2c`.

Without Vivado, `sh tools/area_yosys.sh` prints a yosys estimate per block (Vivado is usually smaller, because it maps wide multiplexers better):

The result is written to `build/area_yosys.txt`; the last run is in section 5.1.

5.1 yosys estimate (synth_xilinx, whole `ssc2_core`)

module (all instances)	count	LUTs	FFs
noc_router (the 6 routers of the mesh)	6	5539	1560
se_sm (state machines)	2	2766	512
hub	1	2096	1780
noc_pkt_tx (packet senders in the nodes)	5	1160	580
xbar	1	1141	792
dma_writer	1	1044	431
ssc_spi (v1)	1	773	241
ssc2_core (glue, core registers, counters)	1	655	409
se_engine (program memory, FIFOs, registers)	1	515	166
ni_se	1	514	184
ssc_i2c (v1)	1	409	111
ssc_fifo (v1 FIFOs)	10	404	106
noc_pkt_rx (packet receivers in the nodes)	6	384	114
ssc_regbank (v1 registers)	1	273	145
ni_i2c / ni_spi / ni_uart	3	546	347
ssc_uart + baud generator (v1)	2	174	78
time base, interrupts, synchronisers	10	160	142
total		**18553**	**7698**

That is 34.9 % of the Zynq-7020's 53 200 LUTs and 7.2 % of its 106 400 flip-flops (yosys estimate, `build/area_yosys.txt`). Grouped by the v2 blocks of slide 22: network (routers + packet helpers) about 7.1k LUTs, serial engine about 3.8k, hub 2.1k, crossbar 1.1k, DMA writer 1.0k, the reused v1 engines about 2.0k.

Compared with v1 (about 2.3k LUTs for the three engines and the bridge), v2 is several times larger, as slide 22 warned ("more logic: the engine, 6 routers, the hub and the DMA writer"); the six routers are the biggest single item. It still uses about a third of the Zynq-7020.

6. NoC scalability

How: `bash noc_study/run_study.sh` (simulation + yosys, ~10 minutes) and, for real LUTs and Fmax, `vivado -mode batch -source noc_study/vivado_study.tcl` (or `windows\5_noc_study_vivado.bat`). The method, the tables and an honest conclusion are in `noc_study/README.md` and `noc_study/results.md`. In short: the NoC costs more area and adds 2 clocks per hop; it matches the crossbar's throughput and its logic depth does not grow with N, so it only gains (on clock speed) at about 16 nodes.